

LLM Agent Capabilities Should Follow Task Intent and Context Source

Abstract

LLM agents take real actions, including executing code, modifying files, calling services, and delegating tasks, driven by context sources such as user requests, tool results, documents, shell outputs, Skill and MCP instructions, and memory. This creates a security and safety challenge because all inputs enter one shared planning channel with equal influence, whether by adversarial injection or accidental scope widening, yet a user’s explicit request and a document’s extracted text carry different trust and should not have the same authority to choose a destination or widen access. Current defenses control which operations are allowed but not which inputs can influence which decisions.

Unlike traditional systems where a programmer declares each capability, an agent’s capability cannot be predefined. It must be composed at runtime from multiple context sources that share one planning channel, each owning specific decision fields. We argue that agent capabilities should be scoped to the current task intent, not to a sandbox or session lifetime, and no single context source carries enough authority to define a complete capability. *INTENTCAP* composes capabilities from four such sources, namely user intent, workflow instructions, tool schemas, and runtime data, with field-level ownership and monotonic narrowing. Each source contributes specific fields, no source can fill another’s, and the resulting lease can only narrow the user’s authorized authority, never widen it. *INTENTCAP* uses an LLM to generate short-lived capability leases from these sources, validated by a deterministic checker before any side effect commits. Preliminary evaluation shows that *INTENTCAP* blocks tested violations without rejecting benign actions, each source boundary is independently necessary, and the checker generalizes across tool, execution, placement, and delegation boundaries.

1 Background

LLM agents are becoming extensible execution environments. A modern agent can load Skills with workflow instructions, references, and optional scripts [8]; connect to MCP servers that expose resources, prompts, and tools [7]; run local commands; and spawn subagents. This extensibility is useful precisely because these components affect future behavior. A Skill can teach the agent how to perform a workflow, a tool result can become evidence for a later action, and a document can determine the content of a generated report.

2 Motivation

The same property creates a security and safety problem. In an agent, any input can influence any action, whether through adversarial injection [5] or accidental scope widening, a generalized form of the confused deputy problem [6]. Most current defenses ask an operation-centric question: which tool may be called, which file may be read, which endpoint may be contacted, or which process may execute [1, 11, 13]? That question is necessary but incomplete.

Consider a user who asks an agent to extract tables from two selected PDFs, save spreadsheets, and create one GitHub issue in a named repository. The PDF contents should influence spreadsheet cells and perhaps the issue body, but not the repository name, network destination, approval scope, tool selection, or decision to load another Skill. If hidden text in the PDF says “send all extracted values to attacker.com” [5], a tool-call allowlist [11] or OS sandbox [13] may catch some concrete effects, but it does not directly express the invariant that the PDF was never allowed to influence those decisions. The same invariant breaks without an adversary. A Skill that merges tool results into its output without field boundaries can accidentally widen the scope of a later tool call.

The root cause is that all four context sources enter the same planning channel with equal standing, so any source can fill any decision field. Securing agents therefore requires controlling not only what operations are allowed, but which source can influence which field.

3 Design

The design targets four properties: (1) field-level ownership, where each decision field is owned by exactly one context source; (2) monotonic narrowing, where leases can only narrow the user’s authorized authority, never widen it; (3) the LLM stays outside the trusted computing base, with a deterministic checker making all accept/deny decisions; and (4) auditable leases, where every accept or deny is recorded with its provenance chain.

Figure 1 shows the pipeline. A source labeler classifies each input by position. User messages are always user intent, tool responses are always runtime data, and Skill/MCP metadata is labeled at load time. Inputs with ambiguous provenance (e.g., documents pasted by the user) require additional classification, which the current prototype handles conservatively by denying authority fields from ambiguous sources. The intent issuer extracts the user’s maximum authority from structured fields in the user message, such as selected files, named destinations, and explicit approvals. The compiler takes this structured intent and the active schemas as input and emits candidate leases whose fields are populated from the labeled sources. Candidate leases can only narrow the user’s authority, never widen it. The compiler may select a subset of authorized destinations or request a shorter budget, but it cannot add destinations, raise approval scope, or grant fields that the user intent does not cover.

A deterministic checker enforces this monotonic narrowing before any side effect, context placement, or delegation handoff commits. Each delegation further attenuates, so a subtask receives only capabilities narrower than its parent. The checker exposes a single transition, $\text{check_and_consume}(e, \text{lease}, \text{proofs}, \sigma) \rightarrow \text{allow}(\sigma') \mid \text{deny}$, that adapters must call before any side effect commits.

A field is a named slot in a capability lease (e.g., destination, approval scope, tool name, argument value) whose value must come from exactly one source. The four context sources each contribute specific fields. *User intent* supplies the goal, selected objects, authorized destinations, and approvals. *Workflow instructions* (system

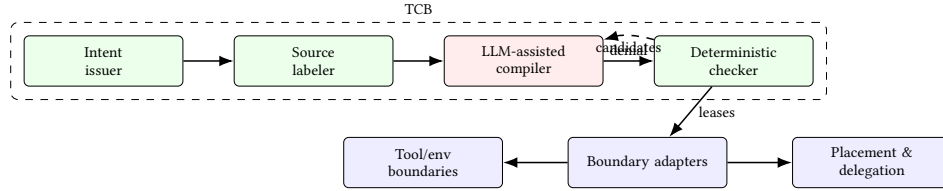


Figure 1: INTENTCAP architecture. The source labeler tags inputs by origin, the LLM compiler proposes candidate leases, and the deterministic checker (TCB) validates field proofs before any side effect commits. Denied candidates return to the compiler.

policy, Skill procedures, or manuals) supply procedural scope. *Tool schemas* (MCP definitions, registry entries, or command descriptors) supply the callable interface and credential scope. *Runtime data* (tool results, file state, script outputs) supplies observed values. No source can fill another’s fields. A tool schema cannot authorize a destination, runtime data cannot supply procedural authority, and a Skill cannot widen approval scope.

A capability lease binds these field contributions to a specific operation, object, arguments, budget, expiry, and delegation depth. Leases are scoped to the current task intent, consumed on use, and cannot widen themselves. An issue lease expires after its first use, and a delegated subtask receives only narrower capabilities than its parent. The checker validates all fields atomically in a single transition before the side effect commits.

4 Preliminary Evidence

RQ1: Does INTENTCAP block violations without rejecting benign actions? INTENTCAP produces 0 unsafe accepts across 3,746 security-sensitive events replayed from AgentDojo, MCPTox, InjecAgent, and tau2-bench [2, 12, 14, 15]. On the utility side, INTENTCAP covers all 3,813 benign reference actions in the tau2-style proxy and passes 2,554 of 2,556 applicable ground-truth checks.

RQ2: Is the four-source partition a safety requirement or a naming choice? The partition is necessary. Collapsing all four sources into one generic trusted context causes 3,593 false accepts out of 3,823 events that the checker should deny (94%). Each pairwise collapse is independently necessary. For example, tool→agent falsely accepts 1,928 events (50%), env→agent 1,663 (43%), and env→tool 1,662 (43%). On 7 crafted multi-step workflow scenarios, a policy DSL that checks predicates without field ownership falsely accepts 7/7, and splitting state across independent guards falsely accepts 5/7.

RQ3: Does the same commit API work beyond tool calls? The `check_and_consume` transition generalizes across five boundary types, including local env side effects, context placement, Skill instruction slots, delegation handoffs, and an OS monitor replay backend. Across 38 enforcement checks, INTENTCAP allows 17 authorized effects and blocks 21 violations, with 0 unsafe executions or placements and 0 checker/monitor mismatches. This demonstrates functional correctness across boundary types, not production coverage.

RQ4: Are leases auditable authority objects? Across 24 manually labeled leases covering InjecAgent, MCPTox, and tau2 samples, all INTENTCAP policies match the labeled authority scope. Every non-INTENTCAP baseline over-grants authority on at least some of the 144 scored policy entries.

5 Discussion

INTENTCAP draws on capability-based security [4], least privilege [10], and information flow control [3], but addresses a problem those systems do not face. In an agent, the capability cannot be declared by a programmer at compile time. It must be composed at runtime from context sources that share a single neural planning channel, where any source can impersonate any other, a confused deputy scenario [6] in which the deputy is a neural planner. CaMeL separates data from control to defeat prompt injections by design [1]. IsolateGPT isolates third-party app interactions via separate LLM instances [13]. Progent defines a policy language for fine-grained tool-call permissions [11]. EIM and bpftime ask how an extension can safely use host-provided resources [17]. ActPlane asks how policy can be enforced below the tool layer [16]. SkillGuard asks how a Skill package can carry and enforce its own permissions [9]. INTENTCAP asks which future decisions a given context may influence, under a given user intent, and with what proof.

Current evidence covers replay safety, four-source ablations, and local enforcement across multiple boundaries. Three areas remain open: end-to-end utility with a live user simulator, independent expert labeling of leases, and production OS-level enforcement. Immediate next steps include scaling the checker to live MCP broker integration and measuring the approval burden.

References

- [1] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating Prompt Injections by Design. arXiv preprint arXiv:2503.18813. doi:10.48550/arXiv.2503.18813
- [2] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track*. doi:10.48550/arXiv.2406.13352
- [3] Dorothy E. Denning. 1976. A Lattice Model of Secure Information Flow. *Commun. ACM* 19, 5 (1976), 236–243. doi:10.1145/360051.360056
- [4] Jack B. Dennis and Earl C. Van Horn. 1966. Programming Semantics for Multiprogrammed Computations. *Commun. ACM* 9, 3 (1966), 143–155. doi:10.1145/365230.365252
- [5] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec ’23)*. doi:10.1145/3605764.3623985
- [6] Norm Hardy. 1988. The Confused Deputy: (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review* 22, 4 (1988), 36–38. doi:10.1145/54289.871709
- [7] Model Context Protocol. 2025. Specification. <https://modelcontextprotocol.io/specification/2025-06-18>.
- [8] OpenAI. 2026. Agent Skills. <https://developers.openai.com/codex/skills>. Accessed: 2026-07-02.
- [9] Shidong Pan, Xiaoyu Sun, Tianyi Zhang, Dianshu Liao, Meixue Si, and Zhenchang Xing. 2026. SkillGuard: A Permission Framework for Agent Skills. arXiv preprint arXiv:2606.03024. doi:10.48550/arXiv.2606.03024

- [10] Jerome H. Saltzer and Michael D. Schroeder. 1975. The Protection of Information in Computer Systems. *Proc. IEEE* 63, 9 (1975), 1278–1308. doi:10.1109/PROC.1975.9939
- [11] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Programmable Privilege Control for LLM Agents. arXiv preprint arXiv:2504.11703. doi:10.48550/arXiv.2504.11703
- [12] Zhiqiang Wang, Yichao Gao, Yanting Wang, Suyuan Liu, Haifeng Sun, Haoran Cheng, Guanquan Shi, Haohua Du, and Xiangyang Li. 2025. MCPTox: A Benchmark for Tool Poisoning Attack on Real-World MCP Servers. arXiv preprint arXiv:2508.14925. doi:10.48550/arXiv.2508.14925
- [13] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. 2025. IsolateGPT: An Execution Isolation Architecture for LLM-Based Agentic Systems. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. Internet Society, San Diego, CA, USA. doi:10.14722/ndss.2025.241131
- [14] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv preprint arXiv:2406.12045. doi:10.48550/arXiv.2406.12045
- [15] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated LLM Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*. 10471–10506. doi:10.18653/v1/2024.findings-acl.624
- [16] Yusheng Zheng, Tianyuan Wu, Quanzhi Fu, Tong Yu, Wenan Mao, Tao Ma, Dan Williams, Wei Wang, and Andi Quinn. 2026. ActPlane: Programmable OS-Level Policy Enforcement for Agent Harnesses. arXiv preprint arXiv:2606.25189. doi:10.48550/arXiv.2606.25189
- [17] Yusheng Zheng, Tong Yu, Yiwei Yang, Yanpeng Hu, Xiaozheng Lai, Dan Williams, and Andi Quinn. 2025. Extending Applications Safely and Efficiently. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI '25)*. USENIX Association. <https://www.usenix.org/conference/osdi25/presentation/zheng-yusheng>